

ACTIVIDAD 1 — Diagnóstico del código

Consigna

Esta actividad será realizada por el equipo evaluador. Trabaje sobre el código proporcionado por el equipo asignado.

Durante esta primera etapa NO modifique el código.

Analice el programa e identifique al menos 5 problemas o aspectos que podrían mejorarse.

Buscar específicamente:

- nombres poco claros;
- funciones demasiado extensas;
- código repetido;
- comentarios innecesarios o insuficientes;
- problemas de indentación o formato;
- responsabilidades mezcladas;
- estructuras condicionales complejas;
- falta de modularización.

Estos aspectos se corresponden con los principios de Clean Code desarrollados en el documento.

Para rellenar esta tabla se analizaron específicamente dos archivos del repo. En cada celda de “Ubicación” se especifica a qué archivo se hace referencia. A continuación pegamos los links de ambos archivos:

<https://github.com/dbetania2/educar-para-transformar/blob/main/src/app/api/admin/courses/route.ts>

<https://github.com/dbetania2/educar-para-transformar/blob/main/src/app/api/admin/solicitudes/%5BrequestId%5D/route.ts>

Nº	Problema	Ubicación	¿Por qué debería mejorarse?
1	Importaciones amontonadas en una sola línea	courses/route.ts (Línea 1)	Hay varios import pegados en la misma línea. Es difícil de leer y no cumple con las reglas básicas de orden y formato de código.
2	Nombre de variable poco claro (candidate)	solicitudes/[requestId]/route.ts (Líneas 9 y 13)	La variable se llama candidate ("candidato"), lo cual no explica qué datos guarda. Es más claro llamarla datosSolicitud o payload.
3	Código repetido	courses/route.ts (Líneas	Las funciones ensureSubject

	(Duplicación)	14 a 33)	y ensureAcademicTerm hacen exactamente los mismos pasos (buscar en la base de datos y si no existe, crear). Si cambia algo, hay que corregirlo dos veces.
4	Función demasiado larga y compleja	solicitudes/[requestId]/route.ts (Líneas 83 a 128)	La función PATCH realiza demasiadas tareas juntas (validar datos, guardar en base de datos, crear usuario y borrar si hay error). Cuesta seguir el hilo y entender qué hace.
5	Falta de comentarios en acciones delicadas	solicitudes/[requestId]/route.ts (Líneas 51 a 60)	Las funciones rollbackManagedUser y cleanupPendingInvitedUser borran o deshacen cambios de usuarios en el sistema, pero no tienen ningún comentario que explique cuándo o por qué se ejecutan.

Pregunta

¿El programa funciona correctamente? ¿Significa esto que necesariamente está bien desarrollado? Explique.

No, que un programa funcione no significa que esté bien desarrollado. Que cumpla su objetivo resuelve únicamente la necesidad inmediata, pero la calidad del código se mide por su legibilidad, mantenibilidad, testabilidad y modularidad. Un programa desordenado o mal estructurado acumula deuda técnica, lo que hace que cualquier cambio futuro sea más costoso, complejo y propenso a errores, dificultando además la colaboración en equipo.

ACTIVIDAD 2 — Mejoramos el código

Devuelva el código al equipo para que, Ahora sí, modifiquen el código.

Aplique al menos cinco mejoras, incluyendo obligatoriamente:

A. Nombres significativos

Cambie variables, funciones o métodos cuyos nombres no permitan comprender claramente su propósito.

El documento propone utilizar nombres que expresen claramente la intención del elemento.

B. Funciones con responsabilidad definida

Identifique al menos una función que realice demasiadas tareas y divídala cuando sea posible.

C. Eliminación de duplicación

Busque código repetido y determine si puede convertirse en una función o componente reutilizable.

D. Legibilidad

Mejore:

- indentación;
- espacios;
- organización;
- estructuras condicionales;
- formato general.

E. Comentarios

Elimine comentarios que simplemente expliquen lo que el código ya expresa claramente y agregue documentación cuando sea necesario explicar una decisión o una situación no evidente.

Mejora	Antes	Después	Justificación
1. Reutilización de componentes en interfaz (ContactForm.tsx)	<pre>/*ANTES*/ <Button variant="default" onClick={() => setOpened(false)}> {actions.review} </Button> <CTAButton icon={<IconSend size={16} />} ...> {actions.confirm} </CTAButton> /*DESPUES*/ <CTAButton variant="default" onClick={() => setOpened(false)}> {actions.review} </CTAButton> <CTAButton icon={<IconSend size={16} />} ...> {actions.confirm} </CTAButton></pre>		Reutilización y Consistencia Visual: Se reemplaza la importación directa del botón nativo de Mantine por el componente atómico reutilizable CTAButton del proyecto. Esto garantiza uniformidad visual y centraliza los estilos de los botones del sistema

2. Nombres significativos en propiedades (Props)
(TeacherCourseActionsMenu.tsx)	<pre>/*ANTES*/ export default function TeacherCourseActionsMenu({ courseHref, gradesHref, attendanceHref, materialsHref, coursesHref, }): TeacherCourseActionsProps) /*DESPUES*/ export default function TeacherCourseActionsMenu_{ courseUrl, gradesUrl, attendanceUrl, materialsUrl, coursesListUrl, }: TeacherCourseActionsProps) </pre>	Nombres Significativos: Los nombres originales generaban confusión entre courseHref (un curso individual) y coursesHref (la lista general) . Renombrarlos a courseUrl y coursesListUrl expresa claramente la intencionalidad y evita errores al consumir el componente
3. Coherencia en el lenguaje del dominio
(TeacherCourseActionsMenu.tsx / features)	<pre>/*ANTES*/ // Mezcla Spanglish: // Carpeta: src/features/alumno/ // Claves e interfaces en inglés, // textos fijos en español: label="Acciones del curso" items={[{ key: "course", label: "Abrir curso" }]} /*DESPUES*/ // Unificación de textos con diccionario/constantes: const ACTIONS_TEXTS = { MENU_LABEL: "Acciones del curso", OPEN_COURSE: "Abrir curso", LOAD_GRADES: "Cargar notas", }; <ActionsMenu label={ACTIONS_TEXTS.MENU_LABEL} ... /> </pre>	Lenguaje Ubicuo y Consistencia: Se eliminan los textos planos "hardcodeados" mezclados en el componente . Centralizar las etiquetas en constantes o diccionarios unifica el lenguaje del dominio y facilita la internacionalización o cambios futuros en el sistema.
4. Eliminación de código duplicado (DRY)
(news.ts)	<pre>/*ANTES*/ // En getPublishedNews: .from("news") .select("id, title, description_html, image_url, file_url, file_name, file_type, is_published, created_at, updated_at") // En getPublishedNewsById: .from("news") .select("id, title, description_html, image_url, file_url, file_name, file_type, is_published, created_at, updated_at") /*DESPUES*/ const NEWS_FIELDS = "id, title, description_html, image_url, file_url, file_name, file_type, is_published, created_at, updated_at"; function getNewsQuery() { return createAdminClient() .from("news") .select(NEWS_FIELDS) .eq("is_published", true); }</pre>	Principio DRY (Don't Repeat Yourself): Ambas funciones duplicaban exactamente la misma lista de columnas seleccionadas de la base de datos . Extraer la consulta base previene inconsistencias si en el futuro se modifica la estructura de la tabla news

5. Nombres claros y reducción de redundancia en tipos (noDocenteDashboard.ts)	<pre> /*ANTES*/ export type NoDocenteStudentRecord = { ... }; export type NoDocenteContactMessageRecord = { ... }; export type NoDocenteRequestRecord = { ... }; export type NoDocenteRouteContext = { ... }; /*DESPUES*/ export type StudentRecord = { ... }; export type ContactMessageRecord = { ... }; export type RequestRecord = { ... }; export type RouteContext = { ... }; </pre>	Nombres sin contexto redundante: Dado que todos los tipos se encuentran ya dentro del módulo noDocenteDashboard.ts, anteponer el prefijo NoDocente a cada tipo resulta redundante . Simplificar los nombres mejora la legibilidad y reduce el ruido visual
--	---	--

Condición

El programa debe continuar funcionando exactamente igual que antes de las modificaciones.

ACTIVIDAD 3 — Refactorización y documentación

Seleccione dos problemas detectados durante el diagnóstico y realice una **refactorización**.

El documento establece que refactorizar consiste en modificar la estructura interna del código sin alterar su comportamiento externo. Entre los ejemplos se encuentran dividir funciones, mejorar nombres, eliminar duplicación, simplificar condicionales y extraer componentes.

Documentación

Agregue documentación mínima al proyecto:

README.md

Debe contener:

1. Nombre del proyecto.
2. Descripción breve.
3. Cómo ejecutar el programa.
4. Funcionalidades principales.
5. Integrantes.

La documentación del proyecto mediante un README es uno de los niveles de documentación mencionados en el material de estudio.

REFACTORIZACIÓN

Refactorización	Problema	Solución aplicada
1) Simplificación de nombres de tipos (src/lib/noDocenteDashb	Los tipos de datos tenían nombres innecesariamente largos y repetitivos porque	Se quitaron los prefijos redundantes dejando nombres más simples como

oard.ts)	todos empezaban con la palabra NoDocente (por ejemplo: NoDocenteStudentRecord, NoDocenteRequestRecord) , a pesar de estar ya dentro del archivo del panel no docente.	StudentRecord y RequestRecord. Esto mejora la legibilidad del código y elimina el ruido visual sin afectar el funcionamiento.
2) Estandarización de componentes de interfaz	El formulario usaba un botón nativo genérico (<Button>) para revisar y el botón propio del proyecto (<CTAButton>) para confirmar, lo que generaba inconsistencia visual.	Se reemplazó el botón genérico por el componente <CTAButton> del proyecto. Esto mejora la reutilización de componentes y asegura que todos los botones de la interfaz mantengan el mismo diseño y estilo.

[README.md](#)

Link al [README.md](#):

https://github.com/dbetania2/educar-para-transformar/blob/mejora-buenas-practicas/NUEVO_README.md

ACTIVIDAD 4 — Registrar el trabajo

🕒 30 minutos

Esta etapa permite aplicar el concepto de control de versiones sobre el código que los estudiantes ya poseen.

Consigna

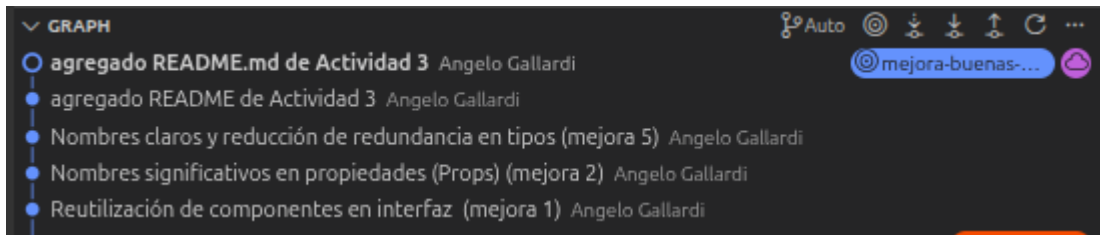
- Crear una rama para las modificaciones:
- mejora-buenas-practicas
- Realizar al menos 3 commits.
- Por ejemplo:
- feat: mejorar estructura del programa
- refactor: mejorar nombres y funciones
- docs: agregar README

Verificar y mostrar al docente.

El estudiante deberá poder mostrar:

- rama creada;
- commits realizados;
- archivos modificados;
- historial del proyecto.

Se adjunta una captura de los commits, incluido el del [README.md](#):



ACTIVIDAD 5 — Code Review

Cada equipo revisará el código de otro compañero.

La revisión debe centrarse en la calidad del código y no en la persona que lo desarrolló. El material propone analizar corrección, legibilidad, estándares, errores, seguridad, rendimiento y pruebas.

Lista de verificación

Marque ✓ o ✗.

Criterio	✓	✗	Observación
Los nombres son claros	✓		-
Las funciones tienen responsabilidades claras	✓		-
No existe duplicación innecesaria	✓		-
El código está correctamente organizado	✓		-
Se aplican convenciones	✓		-
La documentación es suficiente	✓		-
El programa continúa funcionando	✓		-

Finalmente

El revisor deberá indicar:

- **Una fortaleza del código:** La definición estructurada de datos mediante TypeScript (como los tipos para los registros de estudiantes y solicitudes) y la creación de componentes visuales reutilizables (como CTAButton), lo que previene errores de programación y ayuda a mantener un diseño consistente.
- **Una mejora que recomienda:** Evitar redundancias y duplicación de código simplificando los nombres de tipos dentro de sus propios archivos (quitando prefijos innecesarios) y agrupando consultas repetidas de la base de datos en funciones comunes (principio DRY) para facilitar el mantenimiento.

ACTIVIDAD 6 — Cierre y puesta en común

Cada equipo dispone de aproximadamente 1 minuto para responder:

1. **¿Cuál era el principal problema encontrado?** La inconsistencia visual en los botones del formulario y la redundancia en los nombres de los tipos de TypeScript.
2. **¿Qué mejora realizaron?** Estandarizamos la interfaz usando el componente reutilizable <CTAButton> y simplificamos los nombres de los tipos (quitando prefijos innecesarios).
3. **¿Cómo comprobaron que el programa seguía funcionando?** Asegurando que TypeScript compile al 100% sin errores y haciendo pruebas manuales de funcionamiento en el formulario.
4. **¿Qué buena práctica consideran más importante para ese código?** La reutilización de código y la claridad de los nombres para que cualquier compañero del equipo pueda entender y mantener el sistema rápidamente.

Entrega Actividad

Al finalizar las tres horas, cada estudiante/equipo deberá entregar:

- README.md.
- Imagen de Rama de trabajo;
- mínimo 3 commits;
- historial disponible.

Informe breve

Una hoja como máximo con:

1. Problemas identificados por el equipo evaluador:

5 problemas identificados.

2. Mejoras realizadas:

5 mejoras implementadas.

3. Refactorizaciones:

2 refactorizaciones explicadas.

4. Code Review:

observación realizada sobre el código de otro equipo.

5. Reflexión final:

¿Qué diferencia existe entre que un programa "funcione" y que tenga "calidad"?

Co-Evaluación

Para una actividad de solamente tres horas, se realizará una evaluación sencilla por parte del equipo evaluador:

Criterio	Puntaje	Puntaje obtenido
----------	---------	------------------

Diagnóstico del código	20	20
Aplicación de Clean Code	25	22
Refactorización	20	20
Documentación	10	10
Uso correcto de Git/commits	15	15
Code Review	10	10
Total	100	97